

Workshop Overview

01

Cost Estimation

02

**Lab: Execution
Plans**

Cost

Depends on:

- **File organisation**
- **Reduction Factor**
 - Estimates % **portion** of the relation that satisfies the given condition

General Cost Estimation

- Selection:

* If no selection (predicates), RF = 1

- = Result size estimation
- = size of relation * product of **reduction factors**

$$ResultSize = NTuples(R) \prod_{i=1..n} RF_i$$

- Joins (over **k** tables):

- = Product of size of relations * product of **reduction factors**

$$ResultSize = \prod_{j=1..k} NTuples(R_j) \prod_{i=1..n} RF_i$$

Reduction Factor

NKeys: Range of possible values for the specific attribute

Predicate	<u>Reduction Factor</u> Value
attribute = <u>value</u>	$1 / \text{NKeys}(\text{attribute})$
attribute > <u>value</u>	$(\text{High}(\text{attribute}) - \text{value}) / (\text{High}(\text{attribute}) - \text{Low}(\text{attribute}))$
attribute < <u>value</u>	$(\text{value} - \text{Low}(\text{attribute})) / (\text{High}(\text{attribute}) - \text{Low}(\text{attribute}))$
attribute_A = attribute_B	$1 / (\text{Max} (\text{NKeys}(\text{attribute_A}), \text{NKeys}(\text{attribute_B})))$
No info about NKeys / interval	1/10

01

**Single-Relation
Plans**

02

**Multi-Relation
Plans**

Cost Estimation

Indexing Cost

1. Sequential (heap) scan:

- $\text{Cost} = \text{NPages}(\text{R})$

2. Index selection over a primary key (just a single tuple):

- $\text{Cost}(\text{B+Tree}) = \text{Height}(\text{I}) + 1$
 - Height is the index height
- $\text{Cost}(\text{HashIndex}) = \text{ProbeCost}(\text{I}) + 1$
 - $\text{ProbeCost}(\text{I}) \sim 1.2$

3. Clustered index matching ≥ 1 predicates:

$$\text{Cost}(\text{B+Tree}) = (\text{NPages}(\text{I}) + \text{NPages}(\text{R})) * \prod_{i=1..n} RF_i$$

$$\text{Cost}(\text{HashIndex}) = \text{NPages}(\text{R}) * \prod_{i=1..n} RF_i * 2.2$$

4. Non-clustered index matching ≥ 1 predicates:

$$\text{Cost}(\text{B+Tree}) = (\text{NPages}(\text{I}) + \text{NTuples}(\text{R})) * \prod_{i=1..n} RF_i$$

$$\text{Cost}(\text{HashIndex}) = \text{NTuples}(\text{R}) * \prod_{i=1..n} RF_i * 2.2$$

Q1. Single-Relation Plans

Consider a relation with this schema:

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*
- An unclustered hash index on *age*
- A clustered B+ tree index on (*age*, *sal*)

The Employees relation contains 10,000 pages and each page contains 20 tuples. Suppose there are 500 index pages for B+ tree indexes and 500 index pages for hash indexes. There are 40 distinct values of *age*, ranging from 20 to 60, in the relation. Similarly, *sal* ranges from 0 to 50,000 and there are up to 50,000 distinct values. *eid* is a candidate key; its value ranges from 1 to 200,000 and there are 200,000 distinct values.

For each of the following selection conditions, compute the Reduction Factor (selectivity) and the cost of the *cheapest* access path for retrieving all tuples from Employees that satisfy the condition:

Q1. Single-Relation Plans

$$N_{\text{Pages}}(\text{B}^+ \text{ tree index}) = 500$$

$$N_{\text{Pages}}(\text{hash index}) = 500$$

$$N_{\text{Pages}} = 10,000$$

$$N_{\text{Tuples}} = 20 \times 10,000 = 200,000$$

The Employees relation contains 10,000 pages and each page contains 20 tuples. Suppose there are 500 index pages for B+ tree indexes and 500 index pages for hash indexes. There are 40 distinct values of *age*, ranging from 20 to 60, in the relation. Similarly, *sal* ranges from 0 to 50,000 and there are up to 50,000 distinct values. *eid* is a candidate key; its value ranges from 1 to 200,000 and there are 200,000 distinct values.

$$n_{\text{Keys}}(\text{age}) = 40$$

$$n_{\text{Keys}}(\text{sal}) = 50,000$$

$$n_{\text{Keys}}(\text{eid}) = 200,000$$

Q1(a) sal > 20,000

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema:

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*
- An unclustered hash index on *age*
- A clustered B+ tree index on (*age*, *sal*)

Q1(a) sal > 20,000

- Compute the:
 - a. **Reduction Factor (selectivity)**
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Predicate	<u>Reduction Factor</u> Value
attribute = <u>value</u>	1 / NKeys(attribute)
attribute > <u>value</u>	(High(attribute) - <u>value</u>) / (High(attribute) - Low(attribute))
$RF = \frac{High(I) - value}{High(I) - Low(I)} = \frac{50,000 - 20,000}{50,000 - 0} = 0.6$	
attril	
No info	

Q1(a) sal > 20,000

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. **Cost of the cheapest access path**

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema: **What possible access paths are there?**

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*

Non-clustered index matching ≥ 1 predicates:

$$\text{Cost(B+Tree)} = (\text{NPages(I)} + \text{NTuples(R)}) * \prod_{i=1..n} \text{RF}_i$$

- Full table scan:
 - I/O cost = NPages = 10000
- Unclustered B+ tree index on *sal*:
 - Cost = (500 + 200,000) * 0.6
 - = 120,3000

Q1(b) age = 25

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema:

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*
- An unclustered hash index on *age*
- A clustered B+ tree index on (*age*, *sal*)

Q1(b) age = 25

- Compute the:
 - a. **Reduction Factor (selectivity)**
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Predicate	<u>Reduction Factor</u> Value
attribute = <u>value</u>	1 / NKeys(attribute)
attribute > <u>value</u>	$RF = \frac{1}{NKeys(I)} = \frac{1}{40}$
attribute < <u>value</u>	
attribute_A = attribute_B	
No info about NKeys / interval	

Q1(b) age = 25

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. **Cost of the cheapest access path**

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema: **What possible access paths are there?**

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*
- An unclustered hash index on *age*
- A clustered B+ tree index on (*age*, *sal*)

1. Full table scan
2. Unclustered hash index on *age*
3. Clustered B+ tree index on (*age*, *sal*)

Q1(b) age = 25

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. **Cost of the cheapest access path**

1. Full table scan

- I/O cost = NPages = 10000

2. Unclustered hash index on *age*

- = $(200,000) * (1/40) * 2.2$
- = 11,000

3. Clustered B+ tree index on (*age*, *sal*)

- = $(500 + 10,000) * (1/40)$
- = ~ 263

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

3. Clustered index matching ≥ 1 predicates:

$$\text{Cost(B+Tree)} = (\text{NPages(I)} + \text{NPages(R)}) * \prod_{i=1..n} RF_i$$

$$\text{Cost(HashIndex)} = \text{NPages(R)} * \prod_{i=1..n} RF_i * 2.2$$

4. Non-clustered index matching ≥ 1 predicates:

$$\text{Cost(B+Tree)} = (\text{NPages(I)} + \text{NTuples(R)}) * \prod_{i=1..n} RF_i$$

$$\text{Cost(HashIndex)} = \text{NTuples(R)} * \prod_{i=1..n} RF_i * 2.2$$

Q1(c) age > 30

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema:

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*
- An unclustered hash index on *age*
- A clustered B+ tree index on (*age*, *sal*)

Q1(c) age > 30

- Compute the:
 - a. **Reduction Factor (selectivity)**
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Predicate	<u>Reduction Factor</u> Value
attribute = <u>value</u>	1 / NKeys(attribute)
attribute > <u>value</u>	(High(attribute) - <u>value</u>) / (High(attribute) - Low(attribute))
attril	
attribute	$RF = \frac{\text{High}(I) - \text{value}}{\text{High}(I) - \text{Low}(I)} = \frac{60 - 30}{60 - 20} = 0.75$
No info abo	

Q1(c) age > 30

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. **Cost of the cheapest access path**

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema: **What possible access paths are there?**

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*
- An unclustered hash index on *age*
- A clustered B+ tree index on (*age*, *sal*)

Clustered index matching ≥ 1 predicates:

$$\text{Cost(B+Tree)} = (\text{NPages(I)} + \text{NPages(R)}) * \prod_{i=1..n} \text{RF}_i$$

- Full table scan:
 - I/O cost = NPages = 10000
- Clustered B+ tree index on (*age*, *sal*):
 - Cost = (500 + 10,000) * 0.75
 - = 7875

Q1(d) eid = 1000

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema:

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*
- An unclustered hash index on *age*
- A clustered B+ tree index on (*age*, *sal*)

Q1(d) eid = 1000

- Compute the:
 - a. **Reduction Factor (selectivity)**
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Predicate	<u>Reduction Factor</u> Value
attribute = <u>value</u>	$1 / \text{NKeys}(\text{attribute})$
attribute > <u>value</u>	$(\text{High}(\text{attribute}) - \text{value}) / (\text{High}(\text{attribute}) - \text{Low}(\text{attribute}))$
attribute < <u>value</u>	$(\text{value} - \text{Low}(\text{attribute})) / (\text{High}(\text{attribute}) - \text{Low}(\text{attribute}))$
attribute_A = attribute_B	$1 / (\text{Max} (\text{NKeys}(\text{attribute}_A), \text{NKeys}(\text{attribute}_B)))$
No info about NKeys / interval	1/10

$$= 1 / 200,000$$

Q1(d) eid = 1000

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. **Cost of the cheapest access path**

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema: **What possible access paths are there?**

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*

Non-clustered index matching ≥ 1 predicates:

$\text{Cost}(\text{B+Tree}) = (\text{NPages(I)} + \text{NTuples(R)}) * \prod_{i=1..n} \text{RF}_i$

$\text{Cost}(\text{HashIndex}) = \text{NTuples(R)} * \prod_{i=1..n} \text{RF}_i * 2.2$

- Full table scan:
 - I/O cost = NPages = 10000
- Unclustered hash index on *eid*:
 - Cost = (200,000) * (1/200,000) * 2.2
 - = 2.2

Q1(e) $sal > 20,000 \wedge age > 30$

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema:

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
- An unclustered B+ tree index on *sal*
- An unclustered hash index on *age*
- A clustered B+ tree index on (*age*, *sal*)

Q1(e) $sal > 20,000 \wedge age > 30$

- Compute the:
 - a. **Reduction Factor (selectivity)**
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Predicate	<u>Reduction Factor</u> Value
attribute = <u>value</u>	$1 / \text{NKeys}(\text{attribute})$
attribute > <u>value</u>	$(\text{High}(\text{attribute}) - \text{value}) / (\text{High}(\text{attribute}) - \text{Low}(\text{attribute}))$
attribute < <u>value</u>	$(\text{value} - \text{Low}(\text{attribute})) / (\text{High}(\text{attribute}) - \text{Low}(\text{attribute}))$
attribute_A = attribute_B	$1 / (\text{Max}(\text{NKeys}(\text{attribute}_A), \text{NKeys}(\text{attribute}_B)))$
No info about NKeys / interval	1/10

Q1(e) $sal > 20,000 \wedge age > 30$

- Compute the:
 - a. **Reduction Factor (selectivity)**
 - b. Cost of the cheapest access path

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

$$RF_{age} = \frac{High(I) - value}{High(I) - Low(I)} = \frac{60 - 30}{60 - 20} = 0.75$$

$$RF_{sal} = \frac{High(I) - value}{High(I) - Low(I)} = \frac{50,000 - 20,000}{50,000 - 0} = 0.6$$

Q1(e) $sal > 20,000 \wedge age > 30$

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. **Cost of the cheapest access path**

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Consider a relation with this schema: **What possible access paths are there?**

Employees (*eid*: integer, *ename*: string, *sal*: integer, *title*: string, *age*: integer)

Suppose that the following indexes exist:

- An unclustered hash index on *eid*
 - An unclustered B+ tree index on *sal*
 - An unclustered hash index on *age*
 - A clustered B+ tree index on (*age*, *sal*)
- Full table scan
 - Unclustered B+ tree index on *sal*
 - Clustered B+ tree index on (*age*, *sal*)

Q1(e) $sal > 20,000 \wedge age > 30$

- Compute the:
 - a. Reduction Factor (selectivity)
 - b. **Cost of the cheapest access path**

1. Full table scan

- I/O cost = NPages = 10000

2. Unclustered B+ tree index on *sal*

- = $(500 + 200,000) * 0.6$
- = 120,300

3. Clustered B+ tree index on (*age*, *sal*)

- = $(500 + 10,000) * 0.75 * 0.6$
- = 4725

- NPages = 10,000
- NTuples = 200,000
- NPages(B+ tree index) = 500
- NPages(Hash index) = 500
- NKeys(age) = 40
- NKeys(sal) = 50,000
- NKeys(eid) = 200,000

Clustered index matching ≥ 1 predicates:

$$\text{Cost(B+Tree)} = (\text{NPages(I)} + \text{NPages(R)}) * \prod_{i=1..n} RF_i$$

$$\text{Cost(HashIndex)} = \text{NPages(R)} * \prod_{i=1..n} RF_i * 2.2$$

Non-clustered index matching ≥ 1 predicates:

$$\text{Cost(B+Tree)} = (\text{NPages(I)} + \text{NTuples(R)}) * \prod_{i=1..n} RF_i$$

$$\text{Cost(HashIndex)} = \text{NTuples(R)} * \prod_{i=1..n} RF_i * 2.2$$

01

**Single-Relation
Plans**

02

**Multi-Relation
Plans**

Cost Estimation

Cost of Joins

Joins (between relations R and S, R = outer, S = inner) Cost

a. NLJ

i. Tuple-oriented NLJ

$$\text{Cost} = \text{NPages}(\text{R}) + \text{NTuples}(\text{R}) * \text{NPages}(\text{S})$$

ii. Page-oriented NLJ

$$\text{Cost} = \text{NPages}(\text{R}) + \text{NPages}(\text{R}) * \text{NPages}(\text{S})$$

iii. Block-oriented NJL (for block_size B)

$$\text{Cost} = \text{NPages}(\text{R}) + \text{ceil}(\text{NPages}(\text{R}) / (\text{B} - 2)) * \text{NPages}(\text{S})$$

b. Hash Join

$$\text{Cost} = 3 * (\text{NPages}(\text{R}) + \text{NPages}(\text{S}))$$

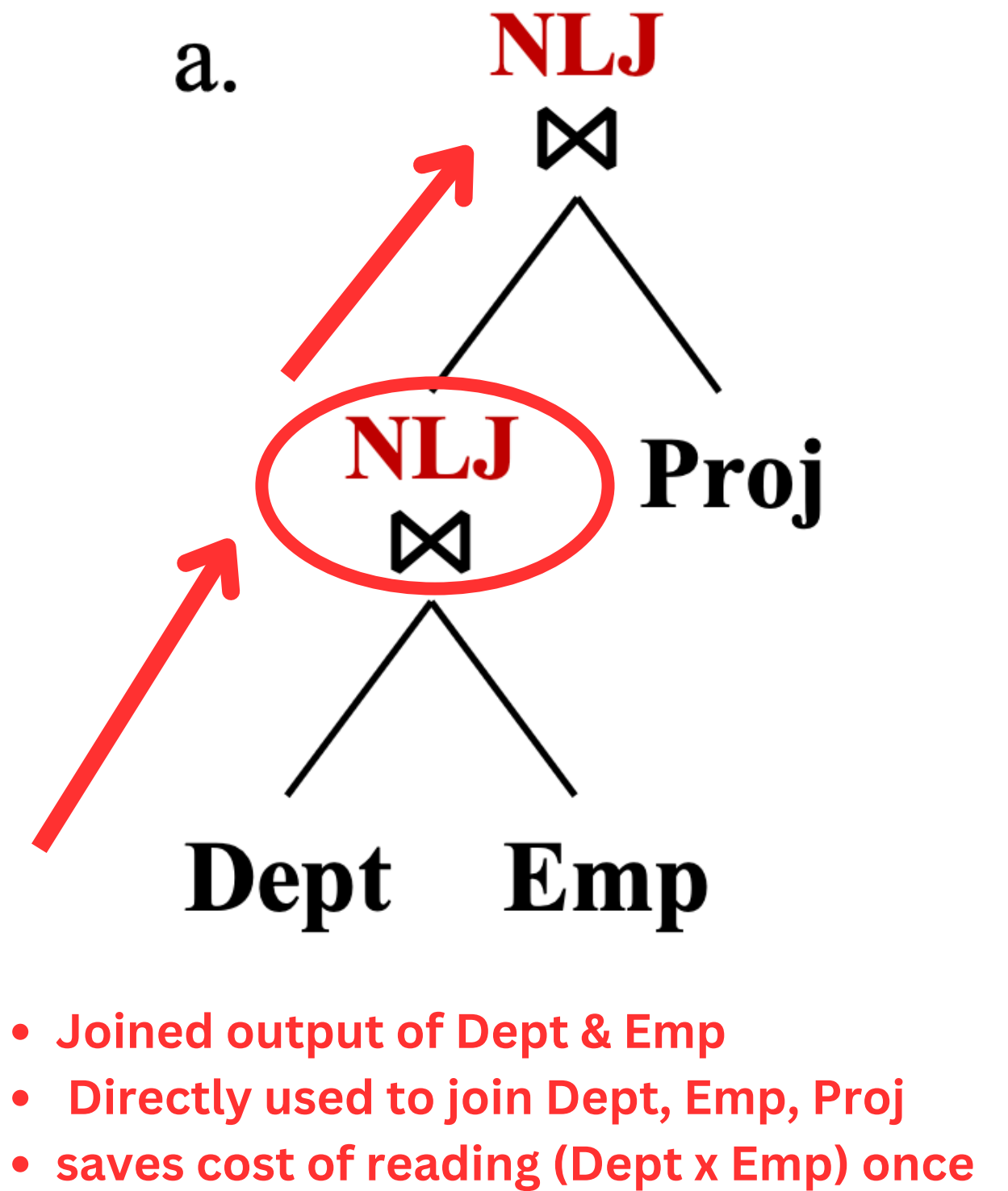
 B = # buffer pages

c. Sort-Merge Join

$$\begin{aligned} \text{Cost}_{\text{SMJ}} = & \text{NPages}(\text{R}) + \text{NPages}(\text{S}) + \\ & 2 * \text{NPages}(\text{R}) * \text{num_passes}(\text{R}) + \\ & 2 * \text{NPages}(\text{S}) * \text{num_passes}(\text{S}) \end{aligned}$$

Pipeline

- **Pipelining** = direct “streaming” in memory of the output of one operation as the input of another operation, without writing the output to disk
- In simpler language:
 - You have to do several joins
 - After doing the first join(s), you can directly use the first output as input for following joins
 - Don't need to reload data from disk



Q2. Multi-Relation Plans

Consider the following schema:

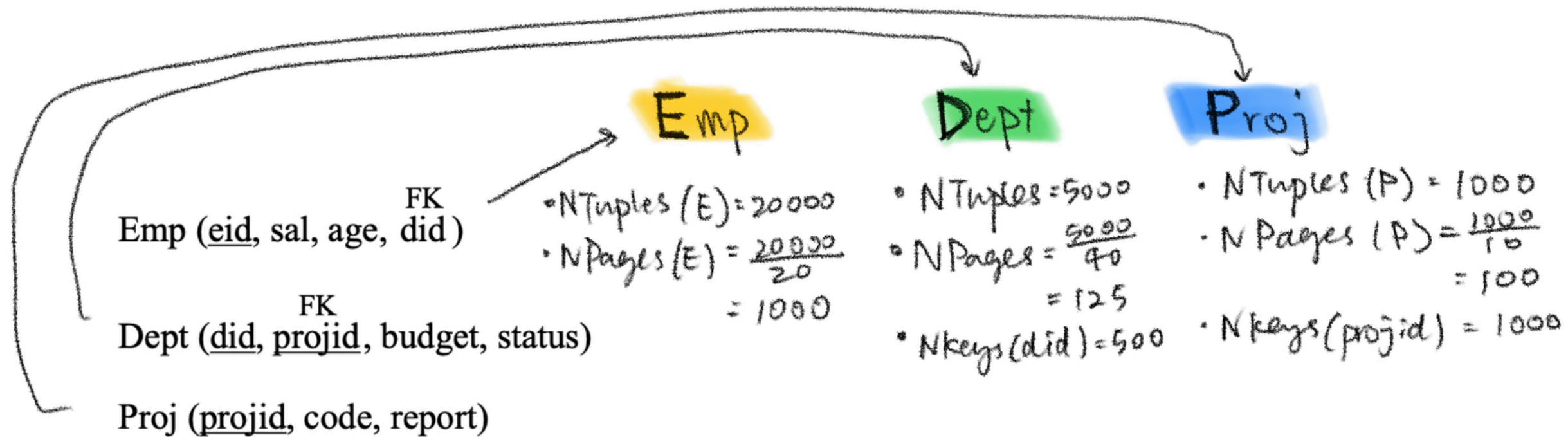
Emp (eid, sal, age, ^{FK}did)

Dept (^{FK}did, projid, budget, status)

Proj (projid, code, report)

The number of tuples in Emp is 20,000 and each page can hold 20 records. The Dept relation has 5000 tuples and each page contains 40 records. There are 500 distinct *dids* in Dept. One page can fit 100 resulting tuples of Dept JOIN Emp. Similarly, Proj has 1000 tuples and each page can contain 10 tuples. Assuming that *projid* is the candidate key of Proj, there can be 1000 unique values for *projid*. Sort-Merge Join can be done in 2 passes. Let's assume that, if we join Proj with Dept, 50 resulting tuples will fit on a page. NLJ in this question means 'Page oriented NLJ'.

Q2. Multi-Relation Plans



The number of tuples in Emp is 20,000 and each page can hold 20 records. The Dept relation has 5000 tuples and each page contains 40 records. There are 500 distinct *dids* in Dept. One page can fit 100 resulting tuples of Dept JOIN Emp. Similarly, Proj has 1000 tuples and each page can contain 10 tuples. Assuming that *projid* is the candidate key of Proj, there can be 1000 unique values for *projid*. Sort-Merge Join can be done in 2 passes. Let's assume that, if we join Proj with Dept, 50 resulting tuples will fit on a page. NLJ in this question means 'Page oriented NLJ'.

- NTuples / page (D x E) = 100
- NTuples / Page (P x D) = 50
- SMJ → 2 passes

Q2. Multi-Relation Plans

Consider the following query:

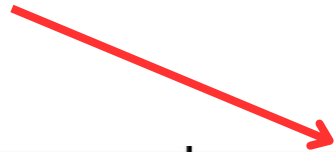
```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```

For this query, estimate the cost of the following plans, focusing on the join order and join types:

- Joins (over k tables):

- = Product of size of relations * product of **reduction factors**

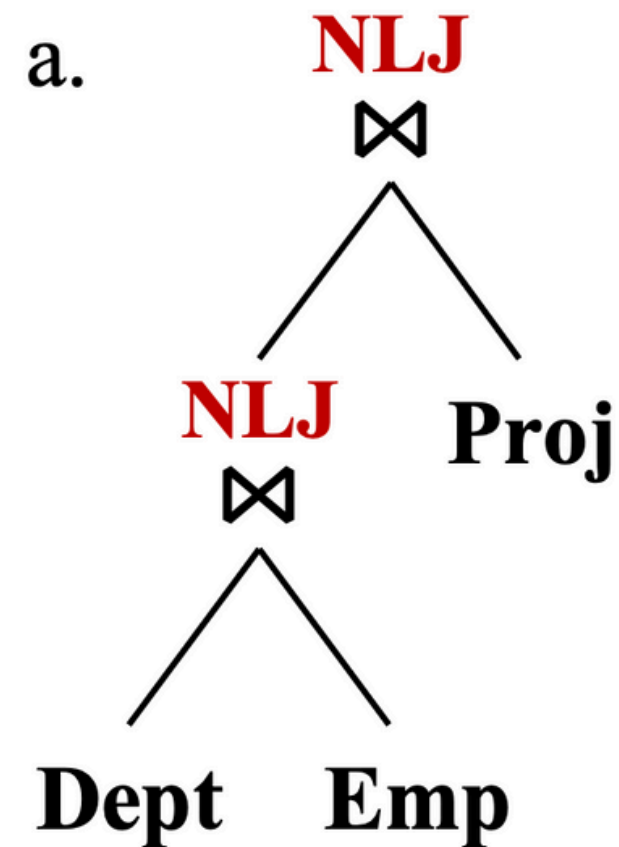
$$ResultSize = \prod_{j=1..k} NTuples(R_j) \prod_{i=1..n} RF_i$$



attribute_A = attribute_B	$1 / (\text{Max}(\text{NKeys}(\text{attribute_A}), \text{NKeys}(\text{attribute_B})))$
---------------------------	--

Q2(a)

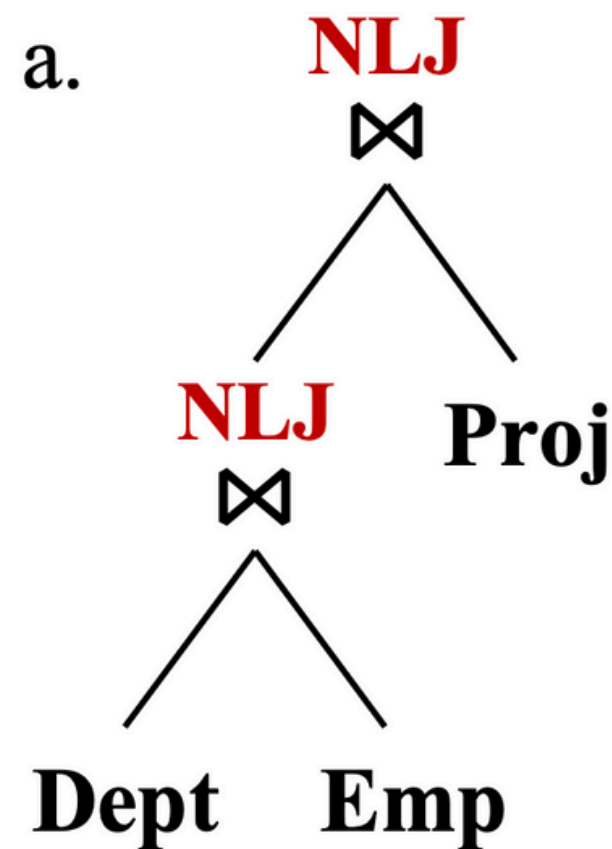
```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```



- NTuples(E) = 20,000
- NPages(E) = 1000
- NTuples(D) = 5000
- NPages(D) = 125
- NKeys(*did*) = 500
- NTuples(P) = 1000
- NPages(P) = 100
- NKeys(*projid*) = 1000
- NTuples/Page(DxE) = 100
- NTuples/Page(PxD) = 50
- SMJ → 2 passes

Q2(a)

```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```



Number of resulting tuples for Dept JOIN Emp

$$\begin{aligned}
 &= \frac{1}{NKeys(I)} \times NTuples(Dept) \times NTuples(Emp) \\
 &= \frac{1}{500} \times 5000 \times 20,000 \\
 &= 200,000 \text{ tuples}
 \end{aligned}$$

$$\text{Number of pages for Dept JOIN Emp} = \frac{200,000}{100} = 2000 \text{ pages}$$

Cost of NLJ = Cost of scanning Dept + Cost to join with Emp =

$$NPages(Dept) + (NPages(Dept) \times NPages(Emp)) =$$

$$125 + 125 \times 1000 = 125,125 \text{ I/O}$$

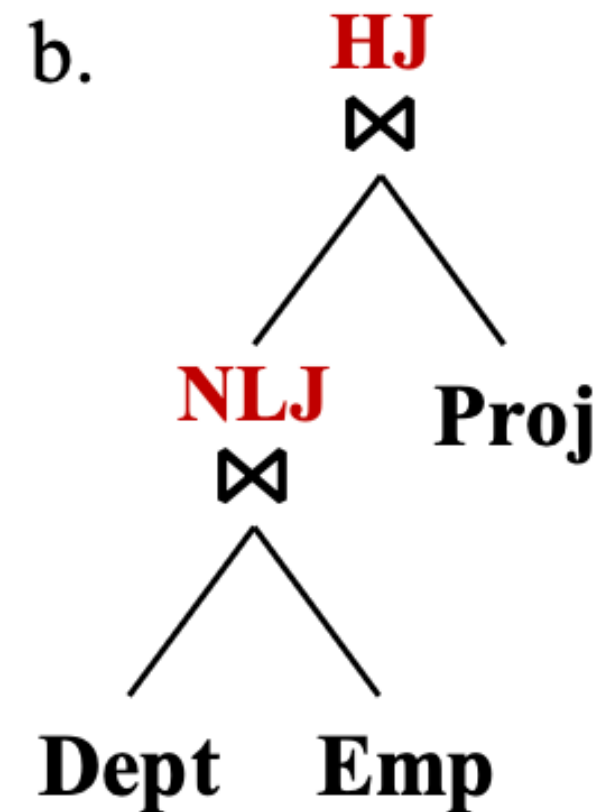
$$\begin{aligned}
 \text{Cost to join with Proj} &= NPages(Dept \text{ JOIN } Emp) \times NPages(Proj) \\
 &= 2000 \times 100 = 200,000 \text{ I/O}
 \end{aligned}$$

$$\text{Total cost} = 125 + 125,000 + 200,000 = \mathbf{325,125 \text{ I/O}}$$

- **NTuples(E) = 20,000**
- **NPages(E) = 1000**
- **NTuples(D) = 5000**
- **NPages(D) = 125**
- **NKeys(*did*) = 500**
- **NTuples(P) = 1000**
- **NPages(P) = 100**
- **NKeys(*projid*) = 1000**
- **NTuples/Page(DxE) = 100**
- **NTuples/Page(PxD) = 50**
- **SMJ → 2 passes**

Q2(b)

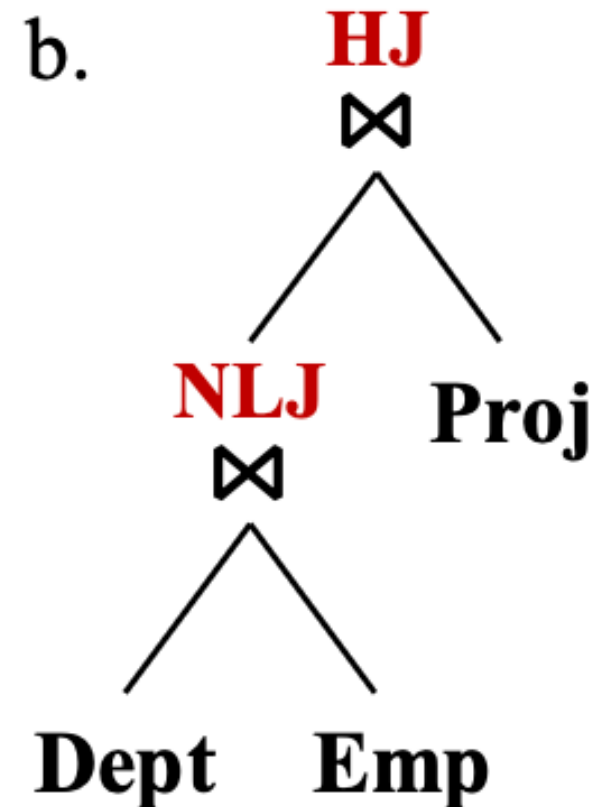
```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```



- NTuples(E) = 20,000
- NPages(E) = 1000
- NTuples(D) = 5000
- NPages(D) = 125
- NKeys(*did*) = 500
- NTuples(P) = 1000
- NPages(P) = 100
- NKeys(projid) = 1000
- NTuples/Page(DxE) = 100
- NTuples/Page(PxD) = 50
- SMJ → 2 passes

Q2(b)

```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```



Number of resulting tuples for Dept JOIN Emp

$$\begin{aligned}
 &= \frac{1}{NKeys(I)} \times NTuples(Dept) \times NTuples(Emp) \\
 &= \frac{1}{500} \times 5000 \times 20,000 \\
 &= 200,000 \text{ tuples}
 \end{aligned}$$

$$\text{Number of pages for Dept JOIN Emp} = \frac{200,000}{100} = 2000 \text{ pages}$$

Cost of NLJ = Cost of scanning Dept + Cost to join with Emp =

$$NPages(Dept) + (NPages(Dept) \times NPages(Emp)) =$$

$$125 + 125 \times 1000 = 125,125 \text{ I/O}$$

Cost to join with Proj = $2 \times NPages(\text{Dept JOIN Emp}) + 3 \times NPages(\text{Proj})$

$$= 2 \times 2000 + 3 \times 100 = 4300 \text{ I/O}$$

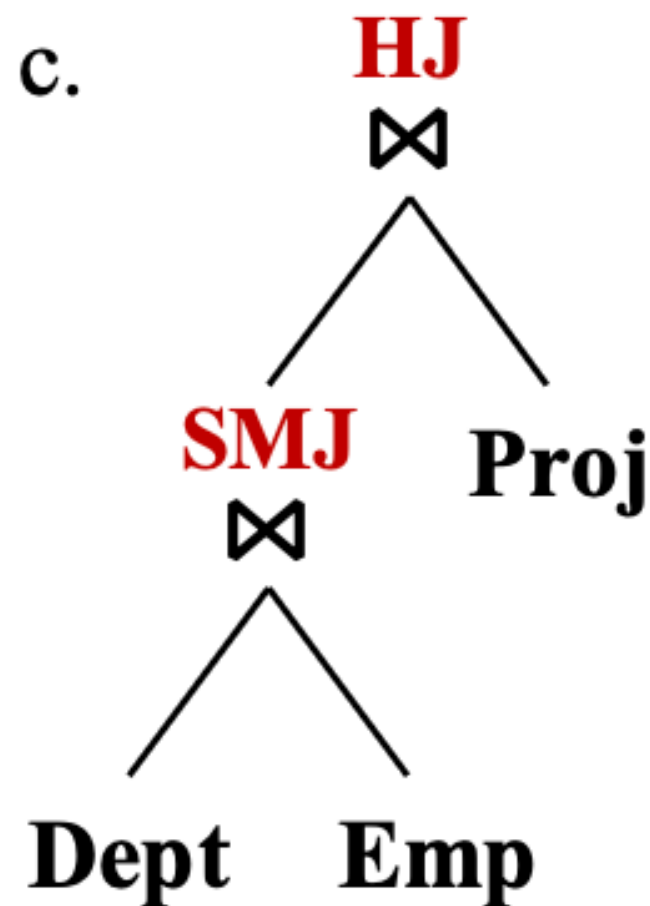
$$\text{Total cost} = 125 + 125,000 + 4300 = 129,425 \text{ I/O}$$

- $NTuples(E) = 20,000$
- $NPages(E) = 1000$
- $NTuples(D) = 5000$
- $NPages(D) = 125$
- $NKeys(did) = 500$
- $NTuples(P) = 1000$
- $NPages(P) = 100$
- $NKeys(projid) = 1000$
- $NTuples/Page(D \times E) = 100$
- $NTuples/Page(P \times D) = 50$
- SMJ $\rightarrow$ 2 passes

Q2(c)

```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```

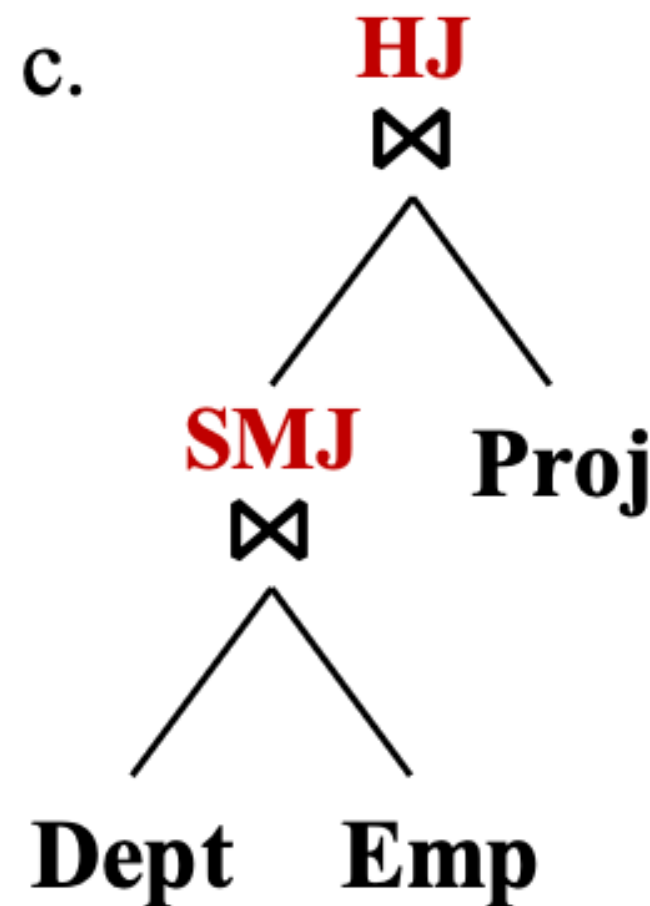
c.



- **NTuples(E) = 20,000**
- **NPages(E) = 1000**
- **NTuples(D) = 5000**
- **NPages(D) = 125**
- **NKeys(*did*) = 500**
- **NTuples(P) = 1000**
- **NPages(P) = 100**
- **NKeys(projid) = 1000**
- **NTuples/Page(DxE) = 100**
- **NTuples/Page(PxD) = 50**
- **SMJ → 2 passes**

Q2(c)

```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```



Number of resulting tuples for Dept JOIN Emp

$$\begin{aligned}
 &= \frac{1}{NKeys(I)} \times NTuples(Dept) \times NTuples(Emp) \\
 &= \frac{1}{500} \times 5000 \times 20,000 \\
 &= 200,000 \text{ tuples}
 \end{aligned}$$

$$\text{Number of pages for Dept JOIN Emp} = \frac{200,000}{100} = 2000 \text{ pages}$$

$$\text{Cost of sorting Dept} = 2 \times NPasses \times NPages(Dept) = 2 \times 2 \times 125 = 500 \text{ I/O}$$

$$\text{Cost of sorting Emp} = 2 \times NPasses \times NPages(Emp) = 2 \times 2 \times 1000 = 4000 \text{ I/O}$$

$$\begin{aligned}
 \text{Cost of joining sorted Dept and Emp} &= NPages(Dept) + NPages(Emp) \\
 &= 125 + 1000 = 1125 \text{ I/O}
 \end{aligned}$$

$$\text{Total cost of SMJ between Dept and Emp} = 500 + 4000 + 1125 = 5625 \text{ I/O}$$

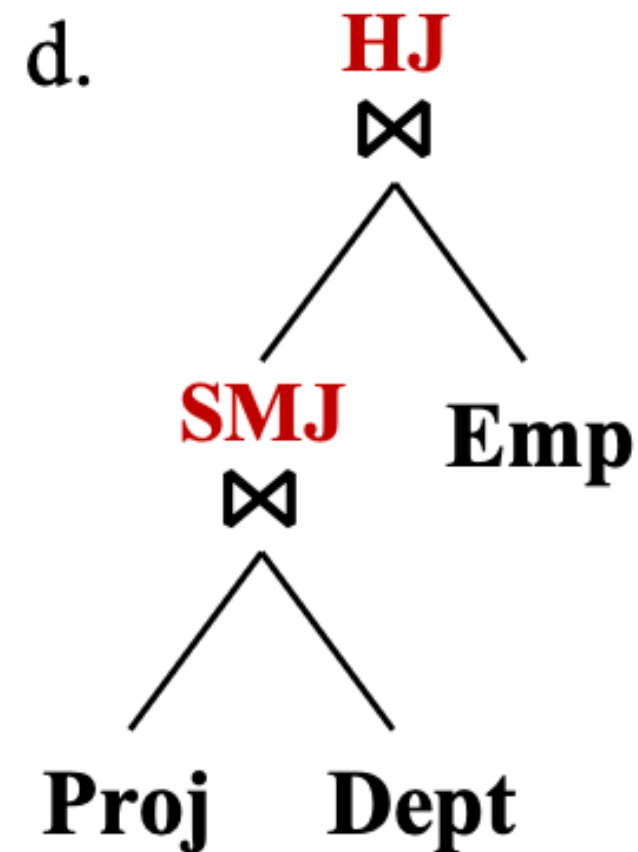
$$\begin{aligned}
 \text{Cost to join with Proj} &= 2 \times NPages(Dept \text{ JOIN } Emp) + 3 \times NPages(Proj) \\
 &= 2 \times 2000 + 3 \times 100 = 4300 \text{ I/O}
 \end{aligned}$$

$$\text{Total cost} = 5625 + 4300 = \mathbf{9925 \text{ I/O}}$$

- $NTuples(E) = 20,000$
- $NPages(E) = 1000$
- $NTuples(D) = 5000$
- $NPages(D) = 125$
- $NKeys(did) = 500$
- $NTuples(P) = 1000$
- $NPages(P) = 100$
- $NKeys(projid) = 1000$
- $NTuples/Page(D \times E) = 100$
- $NTuples/Page(P \times D) = 50$
- SMJ → 2 passes

Q2(d)

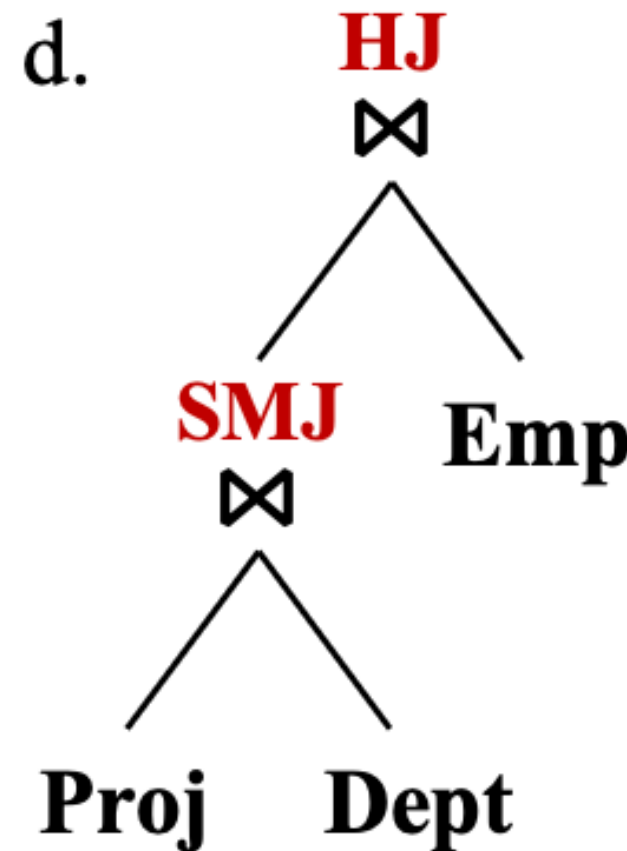
```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```



- NTuples(E) = 20,000
- NPages(E) = 1000
- NTuples(D) = 5000
- NPages(D) = 125
- NKeys(*did*) = 500
- NTuples(P) = 1000
- NPages(P) = 100
- NKeys(projid) = 1000
- NTuples/Page(DxE) = 100
- NTuples/Page(PxD) = 50
- SMJ → 2 passes

Q2(d)

```
SELECT E.eid, D.did, P.projid
FROM Emp AS E, Dept AS D, Proj AS P
WHERE E.did = D.did
      AND D.projid = P.projid;
```



Number of resulting tuples for Proj JOIN Dept

$$\begin{aligned}
 &= \frac{1}{NKeys(I)} \times NTuples(Proj) \times NTuples(Dept) \\
 &= \frac{1}{1000} \times 1000 \times 5000 \\
 &= 5000 \text{ tuples}
 \end{aligned}$$

$$\text{Number of pages for Proj JOIN Dept} = \frac{5000}{50} = 100 \text{ pages}$$

$$\text{Cost of sorting Proj} = 2 \times NPasses \times NPages(Proj) = 2 \times 2 \times 100 = 400 \text{ I/O}$$

$$\text{Cost of sorting Dept} = 2 \times NPasses \times NPages(Dept) = 2 \times 2 \times 125 = 500 \text{ I/O}$$

$$\begin{aligned}
 \text{Cost of joining sorted Proj and Dept} &= NPages(Proj) + NPages(Dept) \\
 &= 100 + 125 = 225 \text{ I/O}
 \end{aligned}$$

$$\text{Total cost of SMJ between Proj and Dept} = 400 + 500 + 225 = 1125 \text{ I/O}$$

$$\begin{aligned}
 \text{Cost to join with Emp} &= 2 \times NPages(Proj \text{ JOIN } Dept) + 3 \times NPages(Emp) \\
 &= 2 \times 100 + 3 \times 1000 = 3200 \text{ I/O}
 \end{aligned}$$

$$\text{Total cost} = 1125 + 3200 = 4325 \text{ I/O}$$

- $NTuples(E) = 20,000$
- $NPages(E) = 1000$
- $NTuples(D) = 5000$
- $NPages(D) = 125$
- $NKeys(did) = 500$
- $NTuples(P) = 1000$
- $NPages(P) = 100$
- $NKeys(projid) = 1000$
- $NTuples/Page(D \times E) = 100$
- $NTuples/Page(P \times D) = 50$
- SMJ → 2 passes

Please complete the take home questions for more practice!

